

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CALIFICACIÓN:

CARRERA DE INGENIERÍA DE SOFTWARE

MODELOS DISCRETOS PARA INGENIERÍA

NRC: 23222

ACTIVIDAD: P3\_Exp1\_Ensayo

APELLIDOS Y NOMBRES: Villarreal Salgado Evelyn Haydée

FECHA: 21/07/2025

APLICACIÓN DEL ALGORITMO DE DIJKSTRA EN UN CASO REAL

**Introducción**

Previamente se investigó un ejemplo de la aplicación de los grafos en la vida real, se escogió el expuesto en el artículo “Teoría de Grafos y Sistemas de Información Geográfica aplicados al Transporte Público de Pasajeros en Resistencia (Argentina).”, en el cual explica como se aplican los grafos en la industria del transporte público para analizar, planificar y mejorar los sistemas de movilizad urbana.

Para lograr el objetivo de optimizar el sistema de transporte público, este es analizado para determinar conectividad y accesibilidad entre las paradas de autobús y comprobar las mejores conexiones para determinar las mejores rutas.

Se puede representar el sistema de trasporte publico mediante en uso de la teoría de grafos y una matriz de incidencia, y determinar rutas más cortas entre distintos puntos mediante el uso del algoritmo Dijkstra.

***Algoritmo Dijkstra***

El algoritmo Dijkstra fue desarrollado en 1959 por Edsger Wybe Dijkstra, este algoritmo nos sirve para encontrar la ruta más corta de un nodo inicial a otro nodo cualquiera, sigue el principio de optimalidad: si en el camino mas corto de un vértice u a un vértice v, se debe parar primero por un vértice w, entonces la sección que va de w a v debe ser el camino más corto de todos loa caminos disponibles que se dirijan de w a v, así se construye el camino de menor costro desde el vértice o nodo de inicio hacia otro. (Torrubia y Terrazas, 2012).

El algoritmo Dijkstra inicia en el nodo que se elija y empieza a analizar el grafo buscando la ruta más corta entre este nodo inicial o también llamado nodo de origen, hacia el resto de nodos del grafo. Realiza el seguimiento del camino de extensión más corta

conocida entre los nodos y en caso de encontrar una ruta mas corta actualiza dicho valor, cuanto ya se encontró la ruta de menor costo el nodo final se marca como visitado. Este proceso se repite hasta que todos los nodos sean visitados (Macias et al., 2023)

### Descripción del caso real expuesto en clase

El artículo investigado previamente usa como área de estudio la ciudad Resistencia, la cual se ubica al sureste de la provincia argentina del Chaco, después de recopilar información necesaria se construyo el grafo identificando rutas y paradas del transporte público, teniendo un total de 48 nodos y 61 arcos o aristas, teniendo en cuenta que:

- Los nodos representan las paradas de autobús.
- Los arcos (aristas) representan los tramos de conexión entre las paradas.

Esta simplificación permite abstraer la red física en una estructura matemática manejable.

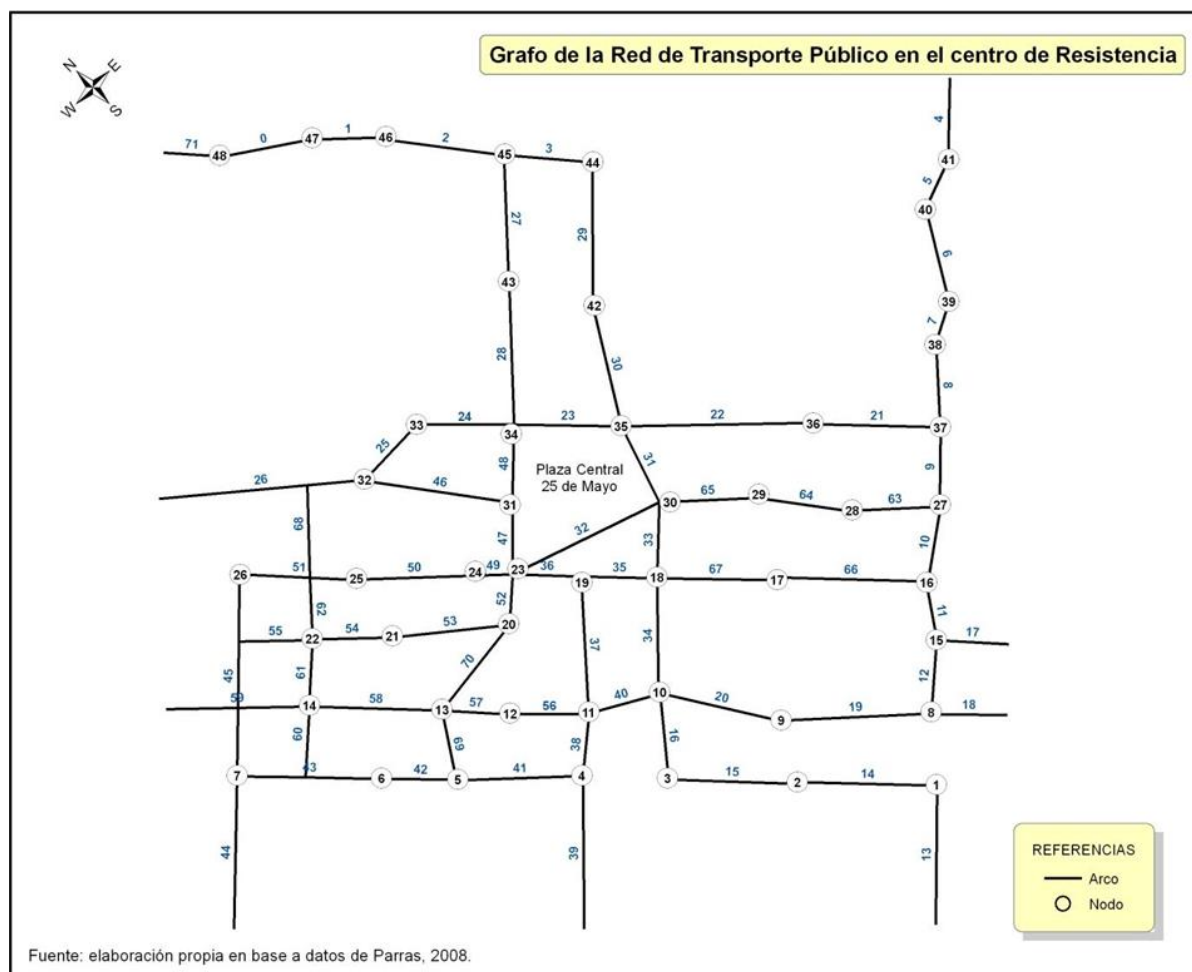


Figura 1. Grafo de la Red del Transporte Público Urbano de Pasajeros en el casco céntrico de la ciudad de Resistencia

## **Aplicación del algoritmo en el caso**

La optimización del transporte público es un factor esencial para satisfacer la necesidad de movilización de toda la población, mejorar la movilidad y reducir la congestión vehicular, para lograr una planificación óptima de rutas y gestión de recursos es necesario utilizar herramientas que permitan analizar y modelar las redes de transporte de manera eficiente. El algoritmo de Dijkstra es una solución idónea para cumplir con este propósito ya que permite calcular de manera precisa la ruta más corta entre dos puntos de un grafo ponderado.

La teoría de los grafos se puede aplicar en este caso ya que una red de transporte de cualquier tipo puede ser representada como un grafo, en el cual los nodos o vértices representan las estaciones, paradas o puntos característicos de la ciudad o país, y las aristas representan los trayectos que conectan a dichos puntos, cada una de estas aristas es ponderada con las magnitudes relevantes para el análisis de la red, tales como el costo de tiempo que toma trasladarse de un punto a otro o la distancia existente entre los mismos. El algoritmo de Dijkstra trabaja bajo la premisa de que todos los pesos de las aristas nunca serán negativos, lo cual siempre se cumple en los sistemas de transporte o de información geográfica, permitiendo la aplicación exitosa del algoritmo.

## **Aspectos computacionales**

Para implementar el algoritmo Dijkstra es necesario representar a la red de transporte como un grafo dirigido o no dirigido, en el cual, cada vértice representa una parada, y cada arista representa la conexión directa entre cada parada, es decir las calles o avenidas, cada arista debe estar asociada a un peso que cuantifique el costo del trayecto, ya sea de tiempo, distancia, costo monetario, etc.

La implementación del algoritmo debe empezar inicializando una estructura en la que se almacene las distancias conocidas entre nodos, la distancia de un nodo hasta sí mismo se le da el valor de cero, mientras que en caso de no existir un arco que una a dos nodos la distancia será infinita. (Restrepo,2004).

Las distancias pueden ser inicializadas de dicha manera mediante una matriz de adyacencia.

Los pasos que sigue un algoritmo de Dijkstra son los siguientes:

- Inicialización: se le asigna una distancia inicial de infinito entre cada nodo, excepto al de origen, el cual debe tener una distancia de cero, se marca a todos los nodos como no visitados.
- Se selecciona un nodo no visitado con la menor distancia
- Para cada nodo vecino del actual, calcula la distancia acumulada a partir del nodo de origen, si se encuentra una distancia menor a la que se registro previamente, esta es actualizada.
- Después de analizar a los vecinos del nodo actual, este se marca como visitado, ya que no se debe pasar dos veces por los nodos ya visitados.
- Se repiten los últimos pasos hasta que todos los nodos hayan sido visitados.

## **Conclusiones**

El uso del algoritmo Dijkstra para el análisis de una red de transporte publico demuestra el gran potencial de las herramientas computacionales para la resolución de problemas, al modelar una ciudad como un grafo es posible transformar a dicho sistema urbano amplio en una estructura matemática más fácil de visualizar y sobre la cual se pueden aplicar algoritmos de optimización con los que se puedan realizar un análisis.

De igual modo, se concluye que el algoritmo Dijkstra además de ser eficaz para la búsqueda de rutas optimas para las redes de transporte público, también es versátil dado a que puede ser adaptado según los criterios y necesidades que tenga el sistema en el cual se deba implementar. Al ofrecer soluciones exactas de forma eficaz se convierte en una herramienta muy útil para los inicios del diseño de la red y las decisiones operativas que se deben tomar en tiempo real.

## **Recomendaciones**

Es recomendable que para el uso de Dijkstra asegurarse de que el grafo con el que se esta trabajando esta definido correctamente mediante listas de adyacencia para representas las conexiones entre nodos, y que a estas se les asignen pesos (distancia, tiempo de traslado de un punto a otro a una velocidad determinada, etc.) que hayan sido calculados previamente de manera precisa o recopilados de una fuente confiable.

## **Referencias**

Restrepo, J. H., & Sánchez, J. J. (2004). Aplicación de la teoría de grafos y el algoritmo de Dijkstra para determinar las distancias y las rutas más cortas en una ciudad. *Scientia et technica*, 10(26), 121-126.

Torrubia, G., & Terrazas, V. (2012). Algoritmo de Dijkstra. Un tutorial interactivo. *VII Jornadas de Enseñanza Universitaria de la Informática (JENUI 2001)*.

Salas, A. (2008). Acerca del algoritmo de dijkstra. *arXiv preprint arXiv:0810.0075*.

Macias, L. R., Rosas, U. O., & Picos, K. (2023). Implementación de algoritmos A\* y Dijkstra para planificación de trayectorias en la navegación autónoma de un robot diferencial. *Revista Aristas*, 242-247.

## **Anexo**

Seudocódigo para el algoritmo Dijkstra

Dijkstra (G, nodo\_origen)

    distancia[nodo\_origen]  $\leftarrow$  0

    Definir cola Q

    Para cada vértice v en G:

        Si  $v \neq$  nodo\_origen:

            distancia[v]  $\leftarrow$  infinito

        prev[v]  $\leftarrow$  indefinido

        agregar v a Q

    Mientras Q no este vacía:

        u  $\leftarrow$  vértice en Q con mínima distancia[u]

        remover u de Q

        Para cada vecino v de u:

            n\_distancia  $\leftarrow$  distancia[u] + tamaño(u, v)

            Si n\_distancia < distancia[v]

                Distancia[v]  $\leftarrow$  n\_distancia

$\text{prev}[v] \leftarrow u$

Retornar distancia[]